

1 Введение в верстку для различных экранов на CSS

В современном мире сайт должен хорошо отображаться на устройствах с различным размером экрана.

Как правило устройства выделяют в четыре класса: компьютеры (desktop), ноутбуки, планшеты (tablet) и телефоны (mobile).

Раньше, в стародавние времена, были известны популярные размеры экранов всех устройств, благо их было не так много. И сайты делали так, чтобы они корректно отображались на известном списке разрешений.

В настоящее время устройств стало так много, что невозможно составить список разрешений, под которые нужно делать сайт. Сейчас считается, что верстка должна корректно отображаться на экране любого размера.

Это достигается перестроением блоков в ключевых точках макета. При этом эти ключевые точки выбираются не под определенные устройства, а под контент сайта - так, чтобы он смотрелся хорошо.

Между ключевыми точками ширина элементов сайта должна быть плавающей, чтобы она подстраивалась под ширину экрана. А в ключевых точках происходит перестроение блоков в другие позиции.

Описанная верстка называется адаптивной или *отзывчивой* (англ. responsive).

2 Введение в медиазапросы на CSS

Существует специальная команда `@media`, которая позволяет выполнять различный код в зависимости от ширины экрана.

В следующем примере некоторый код сработает, если ширина экрана от 300px до 1200px:

```
@media (min-width: 300px) and (max-width: 1200px) {  
    /* некоторый код */  
}
```

В следующем примере некоторый код сработает, если ширина экрана больше 300px:

```
@media (min-width: 300px) {  
    /* некоторый код */  
}
```

В следующем примере некоторый код сработает, если ширина экрана меньше 1200px:

```
@media (max-width: 1200px) {  
    /* некоторый код */  
}
```

Давайте напишем в нашем медиа запросе какой-нибудь код. Например, при определенных размерах экрана покрасим абзацы в красный цвет:

```
@media (min-width: 300px) and (max-width: 1200px) {  
  p {  
    color: red;  
  }  
}
```

```
@media (min-width: 300px) and (max-width: 1200px) {  
  p {  
    color: red;  
  }  
}
```

А теперь покрасим абзацы в различные цвета в зависимости от ширины экрана:

```
@media (max-width: 300px) {  
  p {  
    color: red;  
  }  
}  
@media (min-width: 300px) and (max-width: 900px) {  
  p {  
    color: green;  
  }  
}  
@media (min-width: 900px) and (max-width: 1200px) {  
  p {  
    color: blue;  
  }  
}  
@media (min-width: 1200px) {  
  p {  
    color: orange;  
  }  
}
```

Задача 1

Пусть следующий код применится на экране шириной от 0 до 800px:

```
p {  
  font-size: 20px;  
}
```

А следующий код пусть применится на экране шириной от 800px и больше:

```
p{  
    font-size: 30px;  
}
```

Задача 2

Пусть следующий код применится на экране шириной от 0 до 500px:

```
p{  
    font-size: 15px;  
}
```

Следующий код пусть применится на экране шириной от 500px до 900px:

```
p{  
    font-size: 20px;  
}
```

Следующий код пусть применится на экране шириной от 900px и больше:

```
p{  
    font-size: 30px;  
}
```

3 Перестроение блоков через медиазапросы на CSS

Для начала напомним HTML код наших блоков:

```
<div class="parent">  
    <div class="child">1</div>  
    <div class="child">2</div>  
</div>
```

Теперь давайте напомним стили родителю наших блоков:

```
.parent {  
    display: flex;  
    width: 90%;  
    margin: 50px auto;  
    border: 1px solid red;  
}
```

Теперь напомним стили самим блокам, не указывая тех свойств, которые будут меняться медиазапросами:

```
.child {  
    box-sizing: border-box;  
    padding: 20px;
```

```
border: 1px solid green;
}
```

На большой ширине экрана пусть промежуток между блоками рассчитывается автоматически:

```
@media (min-width: 500px) {
  .parent {
    justify-content: space-between;
  }
}
```

Поставим ширину нашим блокам чуть меньше 50%, чтобы оставить мест под отступ:

```
@media (min-width: 500px) {
  .child {
    width: 49.5%;
  }
}
```

На маленькой ширине экрана поставим наши блоки в колонку:

```
@media (max-width: 500px) {
  .parent {
    flex-direction: column;
  }
}
```

Зададим им нижний отступ:

```
@media (max-width: 500px) {
  .child {
    margin-bottom: 10px;
  }
}
```

Соберем все вместе и получим следующий код:

```
.parent {
  display: flex;
  width: 90%;
  margin: 50px auto;
}
.child {
  box-sizing: border-box;
  padding: 20px;
  border: 1px solid green;
}
```

```

@media (max-width: 500px) {
  .parent {
    flex-direction: column;
  }
  .child {
    margin-bottom: 10px;
  }
}
@media (min-width: 500px) {
  .parent {
    justify-content: space-between;
  }
  .child {
    width: 49.5%;
  }
}

```

Копируйте код и сжимайте ширину окна браузера и смотрите как реагирует браузер

4 Адаптивная плитка без отступов на CSS

Напишем сначала HTML код:

```

<div class="parent">
  <div class="child">1</div>
  <div class="child">2</div>
  <div class="child">3</div>
  <div class="child">4</div>
  <div class="child">5</div>
  <div class="child">6</div>
  <div class="child">7</div>
  <div class="child">8</div>
  <div class="child">9</div>
  <div class="child">10</div>
  <div class="child">11</div>
  <div class="child">12</div>
</div>

```

Давайте теперь добавим стили родителю блоков:

```

.parent {
  display: flex;

```

```
flex-wrap: wrap;  
width: 95%;  
margin: 50px auto;  
}
```

Теперь зададим стили самим блокам, не задавая им ширину:

```
.child {  
  box-sizing: border-box;  
  height: 100px;  
  padding: 20px;  
  border: 1px solid green;  
}
```

Очевидно, что ширина блоков должна быть в процентах, чтобы при изменении экрана блоки плавно меняли свою ширину. При этом в определенных точках экрана мы должны менять ширину блоков так, чтобы в ряд помещалось определенное количество этих блоков.

Давайте напишем код, который поставит четыре блока в ряд:

```
@media (min-width: 1000px) {  
  .child {  
    width: 25%;  
  }  
}
```

А теперь поставим три блока в ряд:

```
@media (min-width: 700px) and (max-width: 1000px) {  
  .child {  
    width: 33.3333%;  
  }  
}
```

А теперь поставим два блока в ряд:

```
@media (min-width: 400px) and (max-width: 700px) {  
  .child {  
    width: 50%;  
  }  
}
```

Один блок в ряд:

```
@media (max-width: 400px) {  
  .child {  
    width: 100%;  
  }  
}
```

Давайте соберем весь код вместе:

```
.parent {
  display: flex;
  flex-wrap: wrap;
  width: 95%;
  margin: 50px auto;
}
.child {
  box-sizing: border-box;
  height: 100px;
  padding: 20px;
  border: 1px solid green;
}

@media (max-width: 400px) {
  .child {
    width: 100%;
  }
}

@media (min-width: 400px) and (max-width: 700px) {
  .child {
    width: 50%;
  }
}

@media (min-width: 700px) and (max-width: 1000px) {
  .child {
    width: 33.3333%;
  }
}

@media (min-width: 1000px) {
  .child {
    width: 25%;
  }
}
```

Посмотрите теперь весь код в браузере как он реагирует на изменение размеров окна браузера.

Задача 1

Сделайте плитку, которая при уменьшении экрана будет сначала четыре элемента в ряд, потом два элемента в ряд, а потом один элемент в ряд.

Задача 2

Сделайте плитку, которая при уменьшении экрана будет сначала шесть элементов в ряд, потом три элемента в ряд, а потом один элемент в ряд.

5 Адаптивная плиточка с отступами на CSS

Для начала сделаем стили родителю блоков:

```
.parent {  
    display: flex;  
    flex-wrap: wrap;  
    width: 95%;  
    margin: 50px auto;  
}
```

Теперь зададим стили самим блокам, не задавая им ширину, но задав нижний марджин в процентах:

```
.child {  
    box-sizing: border-box;  
    height: 100px;  
    padding: 20px;  
    margin-bottom: 1.5%;  
    border: 1px solid green;  
}
```

Теперь напишем код, который поставит четыре блока в ряд, задав соответствующие отступы:

```
@media (min-width: 1000px) {  
    .child {  
        width: calc(100% / 4 - 1.5% * 3 / 4);  
    }  
    .child:nth-child(4n) {  
        margin-right: 1.5%;  
    }  
}
```

А теперь поставим три блока в ряд:

```
@media (min-width: 700px) and (max-width: 1000px) {  
    .child {  
        width: calc(100% / 3 - 1.5% * 2 / 3);  
    }  
}
```



```
.child:not(:nth-child(3n)) {  
    margin-right: 1.5%;  
}  
}
```

А теперь поставим два блока в ряд:

```
@media (min-width: 400px) and (max-width: 700px) {  
    .child {  
        width: calc(100% / 2 - 1.5% * 1 / 2);  
    }  
    .child:not(:nth-child(2n)) {  
        margin-right: 1.5%;  
    }  
}
```

Один блок в ряд:

```
@media (max-width: 400px) {  
    .child {  
        width: 100%;  
    }  
}
```

Давайте соберем весь код вместе:

```
.parent {  
    display: flex;  
    flex-wrap: wrap;  
    width: 95%;  
    margin: 50px auto;  
}  
.child {  
    box-sizing: border-box;  
    height: 100px;  
    padding: 20px;  
    margin-bottom: 1.5%;  
    border: 1px solid green;  
}  
  
@media (max-width: 400px) {  
    .child {  
        width: 100%;  
    }  
}  
  
@media (min-width: 400px) and (max-width: 700px) {  
    .child {
```

```

        width: calc(100% / 2 - 1.5% * 1 / 2);
    }
    .child:nth-child(2n) {
        margin-right: 1.5%;
    }
}
@media (min-width: 700px) and (max-width: 1000px) {
    .child {
        width: calc(100% / 3 - 1.5% * 2 / 3);
    }
    .child:nth-child(3n) {
        margin-right: 1.5%;
    }
}
@media (min-width: 1000px) {
    .child {
        width: calc(100% / 4 - 1.5% * 3 / 4);
    }
    .child:nth-child(4n) {
        margin-right: 1.5%;
    }
}
}

```

Посмотрите теперь весь код в браузере как он реагирует на изменение размеров окна браузера

Задача 1

Сделайте плитку, которая при уменьшении экрана будет иметь сначала восемь элементов в ряд, потом четыре элемента в ряд, потом два элемента в ряд. Пусть отступ между элементами составляет 0.75%.

Задача 2

Модифицируйте предыдущую задачу так, чтобы отступ между элементами составлял фиксированное значение в 20px.

6 Адаптивные размеры шрифтов на CSS

Вы уже знаете, что единицы `rem` считаются относительно размера шрифта корневого элемента, по умолчанию равного `16px`:

```
html {  
    font-size: 16px; /* значение по умолчанию */  
}
```

Такую особенность используют, чтобы массово менять размеры шрифтов при изменении ширины экрана. Давайте посмотрим, как это делается.

Пусть нашим тегам заданы следующие размеры:

```
h1 {  
    font-size: 1.5rem; /* соответствует 24px */  
}  
p {  
    font-size: 1rem; /* соответствует 16px */  
    margin: 2rem; /* соответствует 32px */  
}
```

Давайте сделаем так, чтобы при изменении ширины экрана адаптивно изменялись размеры, заданные через `rem`. Для этого с помощью медиазапросов будем менять размер шрифта корневого элемента:

```
@media (max-width: 600px) {  
    html {  
        font-size: 16px;  
    }  
}  
@media (min-width: 600px) and (max-width: 1200px) {  
    html {  
        font-size: 18px;  
    }  
}  
@media (min-width: 1200px) {  
    html {  
        font-size: 20px;  
    }  
}
```

Задача 1

Дан следующий код:

```
@media (max-width: 300px) {  
    h1 {  
        font-size: 32px;  
    }  
}
```

```
h2 {  
    font-size: 24px;  
}  
p {  
    font-size: 16px;  
}  
}  
@media (min-width: 300px) and (max-width: 900px) {  
    h1 {  
        font-size: 36px;  
    }  
    h2 {  
        font-size: 27px;  
    }  
    p {  
        font-size: 18px;  
    }  
}  
@media (min-width: 900px) {  
    h1 {  
        font-size: 40px;  
    }  
    h2 {  
        font-size: 30px;  
    }  
    p {  
        font-size: 20px;  
    }  
}
```

Упростите приведенный код, используя данную технику.

Практика по медиазапросам

Задача 1

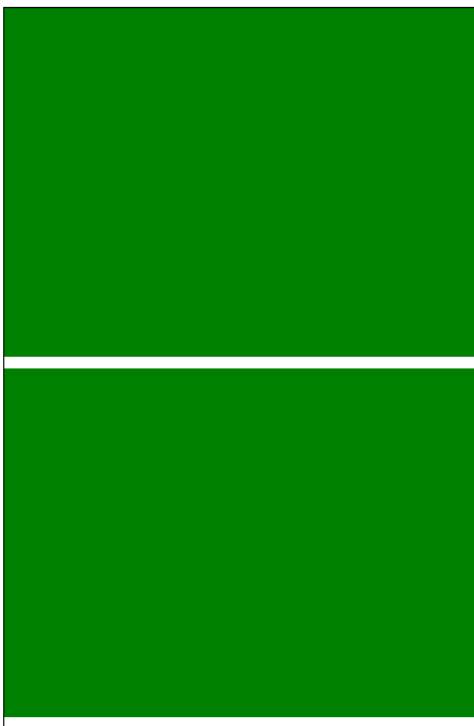
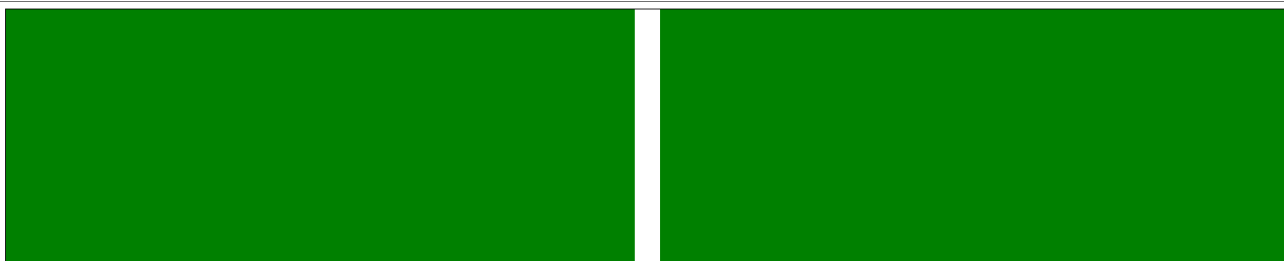
При уменьшении окна браузера
текст меняет цвет

Текст будет разного цвета в зависимости от размера экрана. Точки: 480px, 800px.

Текст будет разного цвета в зависимости от размера экрана. Точки: 480px, 800px.

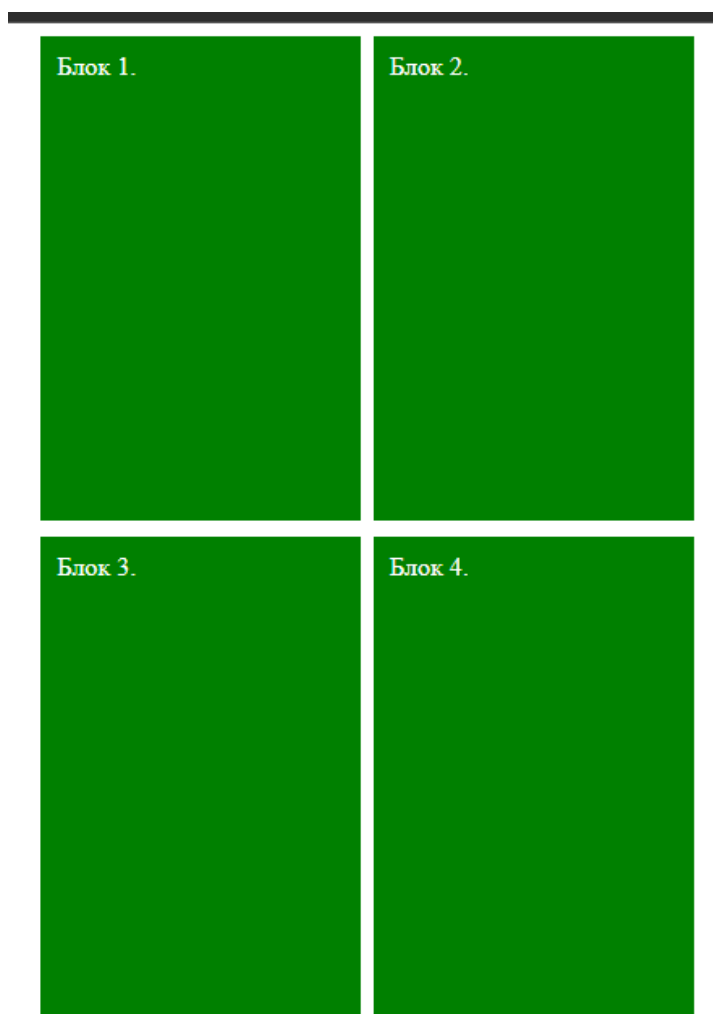
Плиточка на флексах

Повторите страницу Задачу решить с помощью флексов.



Нужно использовать @media screen and (min-width: 800px) {} и @media screen and (max-width: 800px) {}

Повторите страницу . Задачу решить с помощью флексов.



Подсказка (используйте

```
@media screen and (min-width: 800px) {  
  .block {  
    width: 24%;  
  }  
}  
@media screen and (min-width: 480px) and (max-width: 800px) {  
  .block {  
    width: 49%;  
  }  
}  
@media screen and (max-width: 480px) {  
  .block {  
    width: 100%;  
  }  
}
```

И все получится)

Работа с rem

Повторите страницу:

Текст будет разного размера в зависимости от размера экрана. Точки: 480px, 800px.

Текст будет разного размера в зависимости от размера экрана. Точки: 480px, 800px.

Практика

Повторите страницу . Задачу решить с помощью флексов и rem.

При ширине больше 480px изображения встают в ряд по 3 элемента

Заголовок сайта



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam lacinia tortor nisi, sed faucibus mauris lacinia in. Duis sagittis hendrerit dui, eu dignissim dolor porttitor dignissim. Pellentesque auctor dignissim sapien. Pellentesque quis mauris turpis. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Phasellus elementum viverra est, ut accumsan orci dapibus ac. Proin tincidunt consequat nisi, a blandit mauris hendrerit pharetra.



Nulla sit amet diam eget lacus aliquam laculis ac et metus. Quisque eget tortor condimentum, semper feis quis, aliquet quam. Nam luctus laoreet dui nec tristique. Vestibulum in sollicitudin odio. Pellentesque malesuada nulla sit amet congue fermentum. Maecenas vehicula una non libero rhoncus, vel sagittis tortor vehicula. Mauris faucibus sem id fermentum malesuada. Mauris mollis pellentesque sem, vitae cursus nulla posuere a. Vestibulum congue pellentesque elit, et bibendum ipsum aliquet id.



Maecenas tincidunt erat sit amet metus placerat, id porttitor tortor hendrerit. Maecenas at erat dolor. Nam imperdiet diam nec nibh vulputate finibus. Proin ut porta orci. Sed ac placerat lectus. Nunc non erat at dui vehicula sagittis eget a nunc. Ut cursus mauris vitae orci luctus feugiat. Donec ultricies nec nibh ac aliquam.



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam lacinia tortor nisi, sed faucibus mauris lacinia in. Duis sagittis hendrerit dui, eu dignissim dolor porttitor dignissim. Pellentesque auctor dignissim sapien. Pellentesque quis mauris turpis. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Phasellus elementum viverra est, ut accumsan orci dapibus ac. Proin tincidunt consequat nisi, a blandit mauris hendrerit pharetra.



Nulla sit amet diam eget lacus aliquam laculis ac et metus. Quisque eget tortor condimentum, semper feis quis, aliquet quam. Nam luctus laoreet dui nec tristique. Vestibulum in sollicitudin odio. Pellentesque malesuada nulla sit amet congue fermentum. Maecenas vehicula una non libero rhoncus, vel sagittis tortor vehicula. Mauris faucibus sem id fermentum malesuada. Mauris mollis pellentesque sem, vitae cursus nulla posuere a. Vestibulum congue pellentesque elit, et bibendum ipsum aliquet id.



Maecenas tincidunt erat sit amet metus placerat, id porttitor tortor hendrerit. Maecenas at erat dolor. Nam imperdiet diam nec nibh vulputate finibus. Proin ut porta orci. Sed ac placerat lectus. Nunc non erat at dui vehicula sagittis eget a nunc. Ut cursus mauris vitae orci luctus feugiat. Donec ultricies nec nibh ac aliquam.

При минимальной ширине
окна браузера изображения с
текстом встают в колонку

Заголовок сайта



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam lacinia tortor nisi, sed faucibus mauris lacinia in. Duis sagittis hendrerit dui, eu dignissim dolor porttitor dignissim. Pellentesque auctor dignissim sapien. Pellentesque quis mauris turpis. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Phasellus elementum viverra est, ut accumsan orci dapibus ac. Proin tincidunt consequat nisi, a blandit mauris hendrerit pharetra.



Nulla sit amet diam eget lacus aliquam iaculis ac et metus. Quisque eget tortor condimentum, semper felis quis, aliquet quam. Nam luctus laoreet dui nec tristique. Vestibulum in sollicitudin odio. Pellentesque malesuada nulla sit amet congue fermentum. Maecenas vehicula urna non libero rhoncus, vel sagittis tortor vehicula. Mauris faucibus sem id fermentum malesuada. Mauris mollis pellentesque sem, vitae cursus nulla posuere a. Vestibulum congue pellentesque elit, et bibendum ipsum aliquet id.